

Obstáculos para una Llama

Una llama quiere hacer un recorrido en el Altiplano Andino. Tiene un mapa del altiplano en la forma de una grilla de $N \times M$ celdas cuadradas. Las filas del mapa están numeradas de 0 a $N - 1$ de arriba a abajo, y las columnas están numeradas de 0 a $M - 1$ de izquierda a derecha. La celda en el mapa en la fila i y columna j ($0 \leq i < N, 0 \leq j < M$) se denota por (i, j) .

La llama ha estudiado el clima del altiplano y ha descubierto que las celdas en cada fila del mapa tienen la misma **temperatura** y las celdas en cada columna del mapa tienen la misma **humedad**. La llama le ha dado dos arreglos de enteros T y H de longitud N y M respectivamente. Aquí $T[i]$ ($0 \leq i < N$) indica la temperatura en las celdas de la fila i , y $H[j]$ ($0 \leq j < M$) indica la humedad en las celdas en la columna j .

La llama también ha estudiado la flora del altiplano y se ha dado cuenta que una celda está **libre de vegetación** si y solamente si su temperatura es mayor que su humedad, formalmente, $T[i] > H[j]$.

La llama solo puede recorrer el altiplano siguiendo **caminos válidos**. Un camino válido es una secuencia de celdas distintas que satisface las siguientes condiciones:

- Cada par de celdas consecutivas en el camino comparten un lado común.
- Todas las celdas en el camino están libres de vegetación.

Su tarea es responder Q preguntas. Para cada pregunta, se le dan cuatro enteros: L, R, S , y D . Usted debe determinar si existe un camino válido tal que:

- El camino comienza en la celda $(0, S)$ y termina en la celda $(0, D)$.
- Todas las celdas en el camino yacen dentro de las columnas de L a R , inclusive.

Se garantiza que ambos $(0, S)$ y $(0, D)$ están libres de vegetación.

Detalles de Implementación

La primera función que debe implementar es:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : un arreglo de longitud N especificando la temperatura en cada fila.
- H : un arreglo de longitud M especificando la humedad en cada columna.

- Este procedimiento es llamado exactamente una vez para cada caso de prueba, antes de cualquier llamado a `can_reach`.

La segunda función que debe implementar es:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : enteros describiendo una pregunta.
- Esta función se llama Q veces para cada caso de prueba.

Esta función debe devolver `true` si y solamente si existe un camino válido de la celda $(0, S)$ a la celda $(0, D)$, tal que todas las celdas en el camino yacen dentro de las columnas L a R , inclusive.

Restricciones

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ para cada j tal que $0 \leq j < M$.
- $L \leq S \leq R$
- $L \leq D \leq R$
- Ambas celdas $(0, S)$ y $(0, D)$ están libres de vegetación.

Subtareas

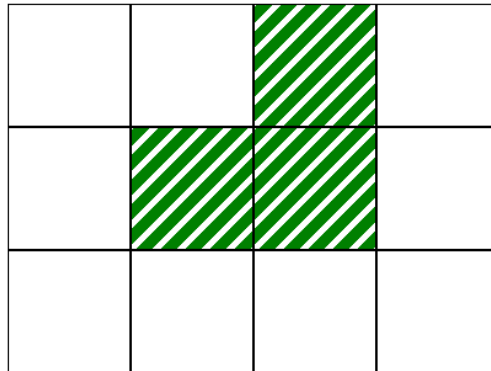
Subtarea	Puntuación	Restricciones adicionales
1	10	$L = 0, R = M - 1$ para cada pregunta. $N = 1$.
2	14	$L = 0, R = M - 1$ para cada pregunta. $T[i - 1] \leq T[i]$ para cada i tal que $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ para cada pregunta. $N = 3$ y $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ para cada pregunta. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ para cada pregunta.
6	17	Sin restricciones adicionales.

Ejemplo

Considere el siguiente llamado:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

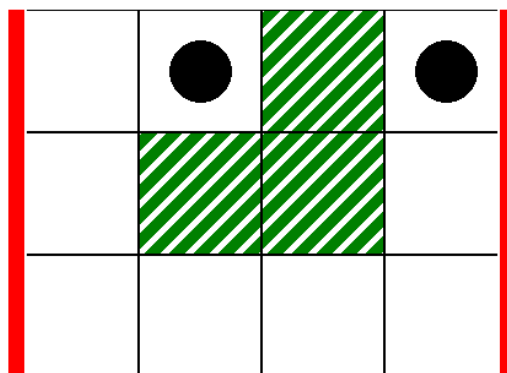
Esto corresponde al mapa en la siguiente imagen, donde las celdas blancas están libres de vegetación:



Como primera pregunta, considere el siguiente llamado:

```
can_reach(0, 3, 1, 3)
```

Esto corresponde a la situación en la siguiente imagen, donde las líneas verticales gruesas indican el rango de las columna de $L = 0$ a $R = 3$, y los discos negros indican las celdas inicial y final:



En este caso, la llama puede llegar de la celda $(0, 1)$ a la celda $(0, 3)$ a través del siguiente camino válido:

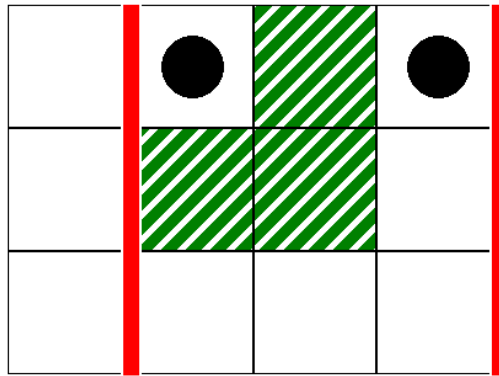
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Por lo tanto, este llamado debería devolver `true`.

Como segunda pregunta, considere el siguiente llamado:

```
can_reach(1, 3, 1, 3)
```

Esto corresponde al escenario en la siguiente imagen:



En este caso, no hay camino válido desde la celda $(0, 1)$ a la celda $(0, 3)$, tal que todas las celdas en el camino yacen dentro de las columnas de la 1 a 3, inclusive. Por lo tanto, este llamado debería devolver `false`.

Evaluador Ejemplo

Formato de entrada:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Aquí, $L[k], R[k], S[k]$ y $D[k]$ ($0 \leq k < Q$) especifican los parámetros para cada llamado a `can_reach`.

Formato de Salida:

```
A[0]
A[1]
...
A[Q-1]
```

Aquí, $A[k]$ ($0 \leq k < Q$) es 1 si el llamado a `can_reach(L[k], R[k], S[k], D[k])` devolvió `true`, y 0 en otro caso.